

Backend Developer Technical Test (Python)

Scenario

You're building a small backend slice of an EPOS (Electronic Point of Sale) for a café. Staff open a tab for a table, add menu items, view totals (including service charge and item-specific VAT), and take payment via an **internal mock payment gateway** implemented in the same codebase. When payment succeeds, the tab is marked as **PAID** and closed.

Tech

- **Language:** Python 3.11+
 - **Framework (choose one):**
 - **Django + Django REST Framework (recommended)**, or
 - **Serverless Framework** (Python on AWS Lambda + API Gateway)
 - **Database:** PostgreSQL (preferred) or MySQL
 - **Testing:** pytest or Django TestCase
-

Functional Requirements

1. Open a tab

POST /api/tabs

JSON

```
{ "table_number": 12, "covers": 2 }
```

Creates a tab in **OPEN** state.

2. Add menu items

POST `/api/tabs/{tab_id}/items`

JSON

```
{ "menu_item_id": 1, "qty": 2 }
```

- Adds an item to the tab.
 - Server calculates:
 - Line subtotal
 - VAT per line (based on item's **vat_rate_percent**)
 - Updates tab totals
-

3. Get tab

GET `/api/tabs/{tab_id}`

Returns:

- List of tab items: name, qty, unit_price_p, vat_rate_percent, vat_p, line_total_p
 - Tab totals: **subtotal_p**, **service_charge_p**, **vat_total_p**, **total_p** (all in pence)
-

4. Create a mock PaymentIntent

POST `/api/tabs/{tab_id}/payment_intent`

- Internally calls your own mock payment gateway class/module.
- Stores the **intent_id** and **client_secret**.

5. Take payment (confirm)

POST `/api/tabs/{tab_id}/take_payment`

- Internally confirms the payment intent.
- If success: mark tab **PAID**, store payment record, set **closed_at**.
- If failure: return **400** or **402** with reason. Tab remains **OPEN**.

Business Rules

- **Service charge**: 10% of subtotal, rounded to pence
- **VAT per line**: `round_to_pence(line_total * vat_rate_percent / 100)`
- **Total**: `subtotal + service charge + VAT`
- **Store all money in pence (integers)**

Suggested Data Model

- **MenuItem**: id, name, unit_price_p, vat_rate_percent
- **Tab**: id, table_number, covers, status, opened_at, closed_at, subtotal_p, service_charge_p, vat_total_p, total_p
- **TabItem**: id, tab_id, menu_item_id, qty, unit_price_p, vat_rate_percent, vat_p, line_total_p
- **Payment**: id, tab_id, payment_intent_id, amount_p, status, created_at

Store price and VAT rate on TabItem to snapshot at time of sale.

Example Menu Items

JSON

```
[
  {"name": "Flat White", "unit_price_p": 350, "vat_rate_percent": 20.0},
  {"name": "Croissant", "unit_price_p": 280, "vat_rate_percent": 0.0},
  {"name": "Iced Tea", "unit_price_p": 300, "vat_rate_percent": 20.0},
  {"name": "Kids Meal", "unit_price_p": 700, "vat_rate_percent": 5.0}
]
```

Mock Payment Gateway (internal)

Python

```
class MockPaymentGateway:
    def create_payment_intent(self, amount_p: int, currency: str = "gbp") -> dict:
        return {"id": "pi_123", "client_secret": "secret_abc", "status": "requires_confirmation"}

    def confirm_payment_intent(self, intent_id: str) -> dict:
        return {"id": intent_id, "status": "succeeded"} # or {"status": "failed", "reason": "..."}
```

- Simulate failures if you like (e.g. if amount ends in 13)
- Keep it fully internal to your app
- `take_payment` must be **idempotent**

Validation & Security

- Validate all input (qty $\geq$ 1, menu item exists and is active, tab must be OPEN)
- Use `X-API-Key` header (e.g., `demo`)

- Return meaningful JSON errors
-

Testing

- Unit tests:
 - VAT + service charge logic
 - Total = subtotal + service + VAT
 - `take_payment` idempotency
 - One end-to-end test:
 - Open tab → Add items → Create PI → Take payment → Tab is marked as PAID
-

Deliverables

Submit a Git repo or a ZIP file with:

-  All source code
 -  Tests
 -  `README.md` with:
 - Set up instructions (Docker or local)
 - Seed data
 - Example API requests
 - Notes or assumptions
-

Time Limit

This test is designed to take **up to 3 hours**, so please keep it scoped accordingly. It's okay if you don't complete everything; we'd look to chat through your approach and what you'd do next during a technical interview.